

GOUVEIA MUSIC

Documentação Técnica do Projeto

Backend, banco de dados, API, regras de negócio, testes e evolução do desenvolvimento

Estado documentado em 20 de setembro de 2026
Projeto de portfólio e aprendizado de desenvolvimento full-stack

Sumário

1. Regras e metodologia desta documentação.....	3
2. Visão geral do Gouveia Music.....	4
3. Histórico do desenvolvimento.....	5
4. Stack tecnológica.....	6
5. Arquitetura do sistema.....	7
6. Estrutura de pastas e arquivos.....	8
7. Banco de dados.....	9
8. CRUD.....	13
9. API e endpoints.....	16
10. Funcionalidades do sistema.....	19
11. Frontend.....	20
12. Backend.....	21
13. Autenticação e autorização.....	22
14. Segurança.....	23
15. Tratamento de erros.....	24
16. Configuração do ambiente.....	25
17. Testes.....	27
18. Problemas encontrados e soluções.....	28
19. Decisões técnicas.....	31
20. Fluxos do sistema.....	32
21. Estado atual do projeto.....	33
22. Funcionalidades pendentes.....	34
23. Próximos passos.....	35
24. Glossário.....	36
25. Conclusão.....	37
A. Apêndice A — server.ts final consolidado.....	38
B. Apêndice B — schema.prisma consolidado.....	39
C. Apêndice C — comandos de diagnóstico e teste usados.....	42

1. Regras e metodologia desta documentação

Esta documentação reconstrói o desenvolvimento do Gouveia Music com base exclusivamente no histórico disponível da conversa. O foco principal é o estado final conhecido; mudanças anteriores são registradas quando ajudam a explicar decisões, erros ou evolução arquitetural.

Classificação	Significado
Implementado	Código ou configuração efetivamente criado no projeto.
Testado	Funcionalidade ou fluxo executado manualmente e confirmado como funcionando.
Parcialmente implementado	Há parte do mecanismo, mas não o fluxo completo.
Pendente	Foi definido como próximo passo ou requisito, mas ainda não foi concluído.
Sugestão / não implementado	Foi proposto durante a conversa, sem confirmação de implementação.
Não identificado	Não existe evidência suficiente no histórico disponível.

Quando um dado não pôde ser confirmado, a expressão “Não identificado no histórico da conversa.” é utilizada literalmente, conforme a regra definida para este documento.

2. Visão geral do Gouveia Music

Gouveia Music é um projeto de portfólio inspirado na experiência de plataformas de streaming de música. O projeto foi usado como primeiro trabalho de portfólio do desenvolvedor e, simultaneamente, como ambiente de aprendizado de programação, banco de dados, APIs, TypeScript, Node.js, Express, PostgreSQL e Prisma.

2.1 Objetivo e finalidade

- Permitir criação de contas e gerenciamento de perfis de usuários.
- Permitir que um usuário evolua para um perfil de artista sem criar uma conta separada.
- Manter catálogo de músicas, artistas, álbuns e gêneros.
- Permitir playlists públicas e privadas com ordenação das músicas.
- Permitir favoritos, histórico de reprodução e relacionamentos sociais.
- Preparar dados que futuramente podem alimentar busca, recomendações e retrospectivas.

2.2 Problema que o sistema pretende resolver

No escopo definido, o sistema reúne gerenciamento de catálogo musical e recursos sociais em uma API própria. A aplicação pretende concentrar perfis, músicas, álbuns, playlists, favoritos, histórico e follows de forma relacional e consistente. Um problema de mercado específico ou público-alvo comercial não foi definido no histórico; o propósito explícito é portfólio e aprendizado.

2.3 Estado atual

Implementado	Backend Node.js + TypeScript + Express organizado em routers, acesso a PostgreSQL via Prisma e CRUDs/fluxos principais testados manualmente.
Pendente	Frontend, autenticação/login com token, busca, player integrado, upload real de arquivos e recomendações ainda não foram implementados.

2.4 Principais componentes

Componente	Estado	Descrição
API backend	Implementado e testado	Express recebe requisições HTTP e encaminha aos routers.
Banco PostgreSQL	Implementado	Banco gouveia_music com estrutura gerenciada pelo Prisma.
Prisma ORM	Implementado	Schema, migrations, client gerado e adapter PostgreSQL.
Frontend	Não implementado	React/Vite foi sugerido, mas não foi construído no histórico atual.
Autenticação	Parcial	Senha é armazenada com bcrypt; login/JWT/middleware não existem ainda.

3. Histórico do desenvolvimento

3.1 Concepção e funcionalidades iniciais

O projeto começou como uma aplicação semelhante a um serviço de streaming musical. As primeiras funcionalidades desejadas incluíram criação de conta, login, busca de músicas, reprodução, playlists personalizadas, recomendações baseadas no histórico, favoritos e possibilidade de seguir artistas e usuários. Uma landing page com animações foi mencionada para uma fase posterior.

3.2 Modelagem de domínio

A modelagem evoluiu por perguntas e respostas até chegar às entidades User, Artist, Song, Album, Genre, Playlist, Favorite, ListeningHistory e às tabelas intermediárias necessárias para relacionamentos N:N. Decisões relevantes incluíram permitir singles sem álbum, álbuns com múltiplos artistas, músicas com múltiplos artistas e gêneros, playlists públicas/privadas e um perfil de artista integrado ao próprio usuário.

3.3 Preparação do ambiente

O ambiente Windows foi configurado com VS Code, Node.js, npm, TypeScript e PostgreSQL. O repositório local principal ficou em C:\Gouveia_Music e o backend em C:\Gouveia_Music\backend. Git foi inicializado e a conta correspondente foi criada.

3.4 Primeiros passos do backend

Antes do Express, foi criado um servidor HTTP simples em TypeScript para compreender compilação e execução. O fluxo `npx tsc --noEmit → npx tsc → node` foi usado para distinguir validação de tipos, compilação e execução. Depois, Express passou a ser a base da API.

3.5 PostgreSQL e Prisma

PostgreSQL foi instalado e o banco `gouveia_music` foi criado. Prisma foi adotado como ORM, usando Prisma 7.10.0 após uma tentativa com Prisma 8 RC. O client foi configurado para geração ESM compatível com o projeto e o adapter `@prisma/adapter-pg` foi usado para conexão.

3.6 Evolução do banco e das migrations

- `create_user_and_artist`
- `create_song_and_song_artist`
- `create_album_and_genre`
- `create_playlist`
- `create_favorite`
- `create_listening_history`
- `create_follow_system`

As migrations foram aplicadas com Prisma e o client foi gerado explicitamente após alterações de schema.

3.7 Construção das rotas

As rotas começaram concentradas em `server.ts`. À medida que o arquivo cresceu, o backend foi refatorado em arquivos específicos de routes: `users`, `artists`, `songs`, `genres`, `albums`, `playlists`, `favorites`, `history` e `follows`. O `server.ts` final ficou responsável somente pela configuração do Express, montagem dos routers, rota raiz, fallback 404 e inicialização.

3.8 Testes e consolidação

Cada seção foi testada manualmente com `Invoke-RestMethod` no PowerShell. Foram verificados casos de sucesso e erros esperados, incluindo duplicidade, IDs inexistentes, payloads vazios, bloqueios de exclusão e a regra de 25% do histórico. Ao final, o usuário confirmou que todos os testes estavam funcionando.

4. Stack tecnológica

Área	Tecnologia	Versão identificada	Uso no projeto
Runtime	Node.js	v24.21.0	Execução do backend compilado.
Gerenciador de pacotes	npm	11.19.0	Dependências e scripts/comandos do projeto.
Linguagem	TypeScript	7.0.2	Código do backend e checagem estática de tipos.
Framework backend	Express	Não identificado no histórico da conversa.	API HTTP e roteamento.
Banco de dados	PostgreSQL	Não identificado no histórico da conversa.	Persistência relacional.
ORM	Prisma	7.10.0	Schema, migrations e operações de banco.
Cliente ORM	@prisma/client	7.10.0	Tipos/client Prisma gerado.
Adapter PostgreSQL	@prisma/adapter-pg	Não identificado no histórico da conversa.	Conexão Prisma ↔ PostgreSQL.
Driver	pg	Não identificado no histórico da conversa.	Driver PostgreSQL usado pelo adapter.
Senha	bcryptjs	Não identificado no histórico da conversa.	Hash de senha com custo 10.
Ambiente	dotenv	Não identificado no histórico da conversa.	Carregamento da DATABASE_URL.
Editor	Visual Studio Code	Não identificado no histórico da conversa.	Desenvolvimento.
Versionamento	Git	Não identificado no histórico da conversa.	Controle de versão local/repositório.

4.1 Frontend

Sugestão / não implementado	React com Vite foi sugerido para o frontend. Nenhuma estrutura de frontend, página, componente ou integração foi implementada no histórico atual.
------------------------------------	---

4.2 API

A API utiliza HTTP e JSON sobre Express. O desenho é REST-like: recursos são expostos por endpoints com GET, POST, PATCH e DELETE. Não há versão de API (/v1) nem namespace /api identificado.

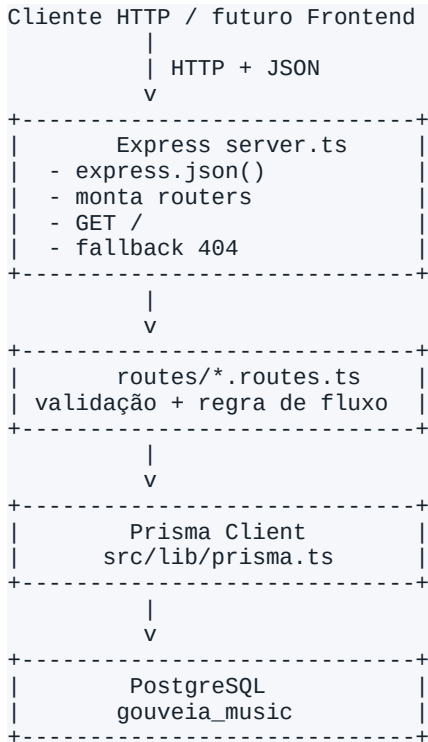
4.3 Autenticação e segurança

bcryptjs está implementado para hash de senhas. JWT, login, logout, cookies, sessões e middleware de autenticação foram discutidos como próximos passos, mas não foram implementados.

5. Arquitetura do sistema

A arquitetura atual é um backend monolítico simples organizado por routers. Não existem camadas separadas de controller, service ou repository. Os próprios arquivos de rota fazem validação, consultam o Prisma e constroem a resposta HTTP.

Diagrama 1 — Arquitetura atual do backend



5.1 Caminho de uma requisição

1. O cliente envia uma requisição HTTP para localhost:3000.
2. `express.json()` interpreta bodies JSON.
3. O Express encontra o router correspondente.
4. O handler valida parâmetros, IDs e body.
5. O handler consulta ou altera dados com Prisma.
6. O Prisma executa a operação no PostgreSQL.
7. O handler retorna JSON ou status sem corpo, conforme o caso.
8. Erros esperados recebem 400, 404 ou 409; falhas não tratadas retornam 500.

5.2 Camadas não existentes

Controllers separados: Não identificado no histórico da conversa. Services: Não identificado no histórico da conversa. Repositories: Não identificado no histórico da conversa. A responsabilidade hoje está concentrada nos routers e no Prisma Client.

6. Estrutura de pastas e arquivos

Estrutura 1 — Estrutura consolidada identificada no backend

```

C:\Gouveia_Music\
├── backend\
│   ├── package.json
│   ├── package-lock.json
│   ├── tsconfig.json
│   ├── .env
│   ├── prisma7.config.ts
│   ├── prisma\
│   │   └── schema.prisma
│   └── src\
│       ├── server.ts
│       ├── lib\
│       │   └── prisma.ts
│       └── routes\
│           ├── users.routes.ts
│           ├── artists.routes.ts
│           ├── songs.routes.ts
│           ├── genres.routes.ts
│           ├── albums.routes.ts
│           ├── playlists.routes.ts
│           ├── favorites.routes.ts
│           ├── history.routes.ts
│           └── follows.routes.ts
│       ├── generated\
│       │   └── prisma\

```

Arquivo/pasta	Responsabilidade
src/server.ts	Configura Express, middleware JSON, rota raiz, routers, 404 e listen na porta 3000.
src/lib/prisma.ts	Carrega dotenv, cria PrismaPg e exporta uma instância de PrismaClient.
src/routes/users.routes.ts	CRUD de usuários e conversão de usuário em artista.
src/routes/artists.routes.ts	Consulta, edição e exclusão protegida de artistas.
src/routes/songs.routes.ts	CRUD de músicas e relação música-gênero.
src/routes/genres.routes.ts	CRUD de gêneros.
src/routes/albums.routes.ts	CRUD de álbuns e associação/desassociação de músicas.
src/routes/playlists.routes.ts	CRUD de playlists e gerenciamento ordenado das músicas.
src/routes/favorites.routes.ts	Favoritar, listar e desfavoritar músicas.
src/routes/history.routes.ts	Registro/listagem de histórico com regra de 25%.
src/routes/follows.routes.ts	Seguir/deixar de seguir artistas e usuários; followers/following.
prisma/schema.prisma	Modelos, relacionamentos, constraints e generator Prisma.
prisma7.config.ts	Configuração Prisma 7, schema/migrations e datasource via DATABASE_URL.
.env	Armazena DATABASE_URL. Credenciais não são documentadas.
src/generated/prisma	Client Prisma gerado.

Pastas controllers, services, repositories, middleware, components, pages e hooks: Não identificado no histórico da conversa.

7. Banco de dados

7.1 Tecnologia e configuração

- Banco: PostgreSQL.
- Nome confirmado: gouveia_music.
- ORM: Prisma 7.10.0.
- Datasource: provider = "postgresql".
- A URL de conexão é lida de DATABASE_URL; credenciais não são reproduzidas.
- Migrations são gerenciadas pelo Prisma.
- Seeds: Não identificado no histórico da conversa.

7.2 User

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
username	String	Sim	Unique	Nome de usuário.
passwordHash	String	Sim	-	Hash da senha.
email	String	Sim	Unique	Email do usuário.
bio	String?	Não	-	Biografia opcional.
profileImageUrl	String?	Não	-	URL da imagem.
createdAt	DateTime	Sim	Default now()	Data de criação.

7.3 Artist

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
userId	Int	Sim	FK + Unique	Liga artista a um único User.
name	String	Sim	-	Nome artístico.
imageUrl	String?	Não	-	Imagem do artista.
bio	String?	Não	-	Biografia.
verified	Boolean	Sim	Default false	Verificado pelo Gouveia Music.
createdAt	DateTime	Sim	Default now()	Criação.

7.4 Song

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
title	String	Sim	-	Título.
duration	Int	Sim	-	Duração em segundos.
coverUrl	String?	Não	-	Capa.
audioUrl	String	Sim	-	Localização/URL do áudio.
createdAt	DateTime	Sim	Default now()	Criação.
albumId	Int?	Não	FK	Álbum opcional; SetNull ao excluir álbum.

7.5 Album

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
title	String	Sim	-	Título.
coverUrl	String?	Não	-	Capa.
releaseDate	DateTime	Sim	-	Data de lançamento.
createdAt	DateTime	Sim	Default now()	Criação.

7.6 Genre

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
name	String	Sim	Unique	Nome único do gênero.

7.7 Playlist

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
userId	Int	Sim	FK	Dono obrigatório.
name	String	Sim	-	Nome.
description	String?	Não	-	Descrição.
coverUrl	String?	Não	-	Capa.
isPublic	Boolean	Sim	Default false	Privacidade.
createdAt	DateTime	Sim	Default now()	Criação.

7.8 Favorite

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
userId	Int	Sim	FK	Usuário.
songId	Int	Sim	FK	Música.
createdAt	DateTime	Sim	Default now()	Favoritado em.
@@unique([userId, songId])	Constraint	-	Unique	Impede favorito duplicado.

7.9 ListeningHistory

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
id	Int	Sim	PK	Autoincremento.
userId	Int	Sim	FK	Usuário.
songId	Int	Sim	FK	Música.
secondsListen	Int	Sim	-	Segundos ouvidos.
playedAt	DateTime	Sim	Default now()	Momento do registro.
@@index([userId, playedAt])	Index	-	Index	Consulta por usuário/data.
@@index([songId])	Index	-	Index	Consulta por música.

7.10 UserArtistFollow

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
userId	Int	Sim	PK composta + FK	Usuário seguidor.
artistId	Int	Sim	PK composta + FK	Artista seguido.
createdAt	DateTime	Sim	Default now()	Data do follow.
@@id([userId, artistId])	Constraint	-	PK	Impede duplicidade.
@@index([artistId])	Index	-	Index	Busca de seguidores do artista.

7.11 UserFollow

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
followerId	Int	Sim	PK composta + FK	Quem segue.
followingId	Int	Sim	PK composta + FK	Quem é seguido.
createdAt	DateTime	Sim	Default now()	Data.
@@id([followerId, followingId])	Constraint	-	PK	Impede duplicidade.
@@index([followingId])	Index	-	Index	Busca followers.

7.12 SongArtist

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
songId	Int	Sim	PK composta + FK	Música.
artistId	Int	Sim	PK composta + FK	Artista.
role	String	Sim	-	Papel; usado "main" por padrão.
@@id([songId, artistId])	Constraint	-	PK	Relação única.

7.13 SongGenre

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
songId	Int	Sim	PK composta + FK	Música.
genreId	Int	Sim	PK composta + FK	Gênero.
@@id([songId, genreId])	Constraint	-	PK	Relação única.

7.14 AlbumArtist

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
albumId	Int	Sim	PK composta + FK	Álbum.
artistId	Int	Sim	PK composta + FK	Artista.
@@id([albumId, artistId])	Constraint	-	PK	Relação única.

7.15 PlaylistSong

Campo	Tipo	Obrigatório	Chave/Constraint	Descrição
playlistId	Int	Sim	PK composta + FK	Playlist.
songId	Int	Sim	PK composta + FK	Música.
position	Int	Sim	-	Posição na playlist.
addedAt	DateTime	Sim	Default now()	Adição.
@@id([playlistId, songId])	Constraint	-	PK	Evita duplicata.
@@index([playlistId, position])	Index	-	Index	Ordenação por playlist.

7.16 Relacionamentos

Diagrama 2 — Visão textual do modelo entidade-relacionamento

```

User 1 ---- 0..1 Artist
User 1 ---- N Playlist
User 1 ---- N Favorite ---- 1 Song
User 1 ---- N ListeningHistory ---- 1 Song
User N ---- N Artist      via UserArtistFollow
User N ---- N User        via UserFollow
Song N ---- N Artist      via SongArtist
Song N ---- N Genre       via SongGenre
Album N ---- N Artist     via AlbumArtist
Playlist N ---- N Song     via PlaylistSong
Album 1 ---- N Song       (Song.albumId opcional)

```

7.17 Regras de integridade e deleção

- Favorite impede duplicidade por userId + songId.
- UserArtistFollow e UserFollow usam chaves compostas para impedir follows duplicados.
- PlaylistSong impede a mesma música de ser inserida duas vezes na mesma playlist.
- Album excluído não exclui Song: albumId é definido como null por SetNull.
- Relações intermediárias usam onDelete: Cascade conforme o schema consolidado.
- A API adiciona bloqueios de exclusão adicionais para preservar regras de negócio, como artista com músicas e última música de um artista.

8. CRUD

CRUD representa Create, Read, Update e Delete. No Gouveia Music, nem toda entidade expõe as quatro operações diretamente: tabelas de relacionamento são manipuladas por endpoints de negócio, favoritos não possuem UPDATE e histórico é tratado como evento, com apenas CREATE e READ.

8.1 Usuário

Operação	Endpoint	Detalhes
CREATE	POST /users	Body: username, email, password. Valida obrigatórios; senha recebe bcrypt.hash(password, 10). Retorna 201. Duplicidade de username/email gera 409.
READ	GET /users; GET /users/:id	Lista ou busca usuário sem expor passwordHash. ID inválido gera 400; inexistente, 404.
UPDATE	PATCH /users/:id	Atualiza username, email, password, bio e profileImageUrl. Somente campos enviados são alterados. Senha é re-hashada. Payload vazio gera 400; duplicidade, 409.
DELETE	DELETE /users/:id	Retorna 204. A rota bloqueia com 409 se o usuário possuir perfil de artista.

8.2 Artista

Operação	Endpoint	Detalhes
CREATE	POST /users/:id/become-artist	Criação indireta e transacional: cria Artist, primeira Song e SongArtist. Exige artistName, title, duration e audioUrl. Usuário já artista gera 409.
READ	GET /artists; GET /artists/:id	Retorna artistas e relações musicais. ID inexistente gera 404.
UPDATE	PATCH /artists/:id	Atualiza name, imageUrl e bio. verified não é aceito nesta rota. Payload vazio gera 400.
DELETE	DELETE /artists/:id	Só permite excluir artista sem músicas relacionadas; caso contrário 409.

8.3 Música

Operação	Endpoint	Detalhes
CREATE	POST /songs	Exige title, duration > 0, audioUrl e ao menos um artista. Confirma todos os artistas antes de criar. Retorna 201.
READ	GET /songs; GET /songs/:id	Inclui artistas, álbum e gêneros. Inexistente: 404.
UPDATE	PATCH /songs/:id	Atualiza title, duration, audioUrl e coverUrl. Relações são tratadas separadamente.
DELETE	DELETE /songs/:id	Bloqueia com 409 se a música for a única música de qualquer artista relacionado; caso contrário 204.

8.4 Gênero

Operação	Endpoint	Detalhes
CREATE	POST /genres	Exige name; aplica trim. Nome duplicado gera 409.

Operação	Endpoint	Detalhes
READ	GET /genres; GET /genres/:id	Lista alfabeticamente e permite consulta individual com músicas relacionadas.
UPDATE	PATCH /genres/:id	Atualiza name; vazio é inválido; duplicidade gera 409.
DELETE	DELETE /genres/:id	Bloqueia gênero que ainda tenha músicas relacionadas; caso contrário 204.

8.5 Álbum

Operação	Endpoint	Detalhes
CREATE	POST /albums	Exige title, releaseDate válida e artistIds não vazio; confirma todos os artistas; cria AlbumArtist aninhado.
READ	GET /albums; GET /albums/:id	Inclui artistas e músicas; consulta individual inclui também gêneros e artistas das músicas.
UPDATE	PATCH /albums/:id	Atualiza title, coverUrl e releaseDate.
DELETE	DELETE /albums/:id	Remove álbum com 204; músicas permanecem e albumId torna-se null.

8.6 Playlist

Operação	Endpoint	Detalhes
CREATE	POST /playlists	Exige userId válido e name; isPublic precisa ser boolean se informado. O dono precisa existir. Retorna 201.
READ	GET /playlists; GET /playlists/:id	Inclui dono e músicas ordenadas por position.
UPDATE	PATCH /playlists/:id	Atualiza name, description, coverUrl e isPublic.
DELETE	DELETE /playlists/:id	Remove playlist; relações PlaylistSong são removidas em cascata; músicas permanecem.

8.7 Favorito

Operação	Endpoint	Detalhes
CREATE	POST /users/:id/favorites	Exige songId e existência de usuário/música. Duplicata gera 409.
READ	GET /users/:id/favorites	Lista favoritos mais recentes primeiro, incluindo relações da música.
UPDATE	Não existe	Não se aplica: favorito é presença/ausência da relação.
DELETE	DELETE /users/:userId/favorites/:songId	Remove a relação; inexistente gera 404; sucesso 204.

8.8 ListeningHistory

Operação	Endpoint	Detalhes
CREATE	POST /users/:id/history	Registra somente se secondsListen atingir pelo menos $\text{ceil}(\text{duration} \times 0,25)$. Abaixo disso responde recorded:false sem inserir.
READ	GET /users/:id/history	Lista reproduções mais recentes primeiro.
UPDATE	Não existe	Histórico foi tratado como evento imutável.

Operação	Endpoint	Detalhes
DELETE	Não existe	Não identificado no histórico da conversa.

8.9 Follows

Operação	Endpoint	Detalhes
CREATE	POST .../following/artists/:artistId; POST .../following/users/:followingId	Cria follow de artista ou usuário. Duplicidade gera 409; auto-follow de usuário gera 400.
READ	GET following/followers	Lista artistas seguidos, followers de artista, usuários seguidos e followers de usuário.
UPDATE	Não existe	Não se aplica ao modelo atual.
DELETE	DELETE endpoints correspondentes	Remove follow; relação inexistente gera 404; sucesso 204.

9. API e endpoints

9.1 Convenções gerais

- Base local utilizada nos testes: `http://localhost:3000`.
- Bodies são JSON quando presentes; `express.json()` processa o conteúdo.
- Autenticação: não implementada. Nenhum header Authorization é exigido atualmente.
- Paginação e query parameters: Não identificados no histórico da conversa.
- Códigos observados: 200, 201, 204, 400, 404, 409 e 500.

Método	Endpoint	Descrição	Parâmetros	Body	Status
GET	/	Health/raiz	-	-	200
GET	/users	Lista usuários	-	-	200
GET	/users/:id	Busca usuário	id	-	200/400/404
POST	/users	Cria usuário	-	username,email,password	201/400/409/500
PATCH	/users/:id	Atualiza usuário	id	campos parciais	200/400/404/409/500
DELETE	/users/:id	Remove usuário não-artista	id	-	204/400/404/409/500
POST	/users/:id/become-artist	Converte usuário em artista	id	artistName,title,duration,audioUrl,coverUrl?	201/400/404/409/500
GET	/artists	Lista artistas	-	-	200
GET	/artists/:id	Busca artista	id	-	200/400/404
PATCH	/artists/:id	Atualiza artista	id	name,imageUrl,bio	200/400/404
DELETE	/artists/:id	Remove artista sem músicas	id	-	204/400/404/409
GET	/songs	Lista músicas	-	-	200
GET	/songs/:id	Busca música	id	-	200/400/404
POST	/songs	Cria música	-	title,duration,audioUrl,coverUrl?,artists[]	201/400/404/500
PATCH	/songs/:id	Atualiza música	id	title,duration,audioUrl,coverUrl	200/400/404
DELETE	/songs/:id	Remove música com proteção	id	-	204/400/404/409
POST	/songs/:id/genres	Relaciona gêneros	id	genreIds[]	200/400/404
GET	/genres	Lista gêneros	-	-	200
GET	/genres/:id	Busca gênero	id	-	200/400/404
POST	/genres	Cria gênero	-	name	201/400/409
PATCH	/genres/:id	Atualiza gênero	id	name	200/400/404/409
DELETE	/genres/:id	Remove gênero sem uso	id	-	204/400/404/409
GET	/albums	Lista álbuns	-	-	200
GET	/albums/:id	Busca álbum	id	-	200/400/404
POST	/albums	Cria álbum	-	title,releaseDate,artistIds[],coverUrl?	201/400/404
PATCH	/albums/:id	Atualiza álbum	id	title,coverUrl,releaseDate	200/400/404
DELETE	/albums/:id	Remove álbum	id	-	204/400/404
POST	/albums/:id/songs	Adiciona músicas	id	songIds[]	200/400/404
DELETE	/albums/:albumId/songs/:songId	Remove música do álbum	albumId,songId	-	200/400/404
GET	/playlists	Lista playlists	-	-	200
GET	/playlists/:id	Busca playlist	id	-	200/400/404
POST	/playlists	Cria playlist	-	userId,name,description?,coverUrl?,isPublic?	201/400/404

Método	Endpoint	Descrição	Parâmetros	Body	Status
PATCH	/playlists/:id	Atualiza playlist	id	name,description,coverUrl,isPublic	200/400/404
DELETE	/playlists/:id	Remove playlist	id	-	204/400/404
POST	/playlists/:id/songs	Adiciona música ordenada	id	songId	201/400/404/409
DELETE	/playlists/:playlistId/songs/:songId	Remove e reordena	playlistId,songId	-	200/400/404
GET	/users/:id/favorites	Lista favoritos	id	-	200/400/404
POST	/users/:id/favorites	Favorita música	id	songId	201/400/404/409
DELETE	/users/:userId/favorites/:songId	Desfavorita	userId,songId	-	204/400/404
GET	/users/:id/history	Lista histórico	id	-	200/400/404
POST	/users/:id/history	Registra reprodução	id	songId,secondsListen	200/201/400/404
GET	/users/:id/following/artists	Artistas seguidos	id	-	200/400/404
GET	/artists/:id/followers	Seguidores do artista	id	-	200/400/404
POST	/users/:userId/following/artists/:artistId	Segue artista	userId,artistId	-	201/400/404/409
DELETE	/users/:userId/following/artists/:artistId	Deixa de seguir artista	userId,artistId	-	204/400/404
GET	/users/:id/following/users	Usuários seguidos	id	-	200/400/404
GET	/users/:id/followers	Seguidores do usuário	id	-	200/400/404
POST	/users/:followerId/following/users/:followingId	Segue usuário	followerId,followingId	-	201/400/404/409
DELETE	/users/:followerId/following/users/:followingId	Deixa de seguir usuário	followerId,followingId	-	204/400/404

9.2 Exemplos de requisição e resposta

Exemplo 1 — Criação de usuário usada nos testes

```
Invoke-RestMethod `
-Uri http://localhost:3000/users `
-Method POST `
-ContentType "application/json" `
-Body '{
  "username":"teste",
  "email":"teste@gouveiamusic.com",
  "password":"123456"
}'
```

Exemplo 2 — Histórico abaixo do limiar de 25%

```
{
  "recorded": false,
  "message": "A reprodução ainda não atingiu 25% da música",
  "secondsListen": 40,
  "minimumSeconds": 58,
  "songDuration": 230
}
```

Exemplo 3 — Erro de conflito ao duplicar música em playlist

```
{
  "message": "A música já está nesta playlist"
}
```

9.3 Documentação individual por grupo

9.3.1 Usuários

Os endpoints de usuário validam IDs, não retornam passwordHash e usam P2002 para conflitos de username/email. POST /users cria o hash bcrypt; PATCH também reprocessa password quando enviado. DELETE é bloqueado para usuários que já possuem Artist.

9.3.2 Artistas

A criação não é feita por POST /artists; ocorre por /users/:id/become-artist em uma transação. PATCH não aceita verified. DELETE exige que o artista não possua músicas.

9.3.3 Músicas

POST exige ao menos um artista existente. PATCH altera somente dados da música. DELETE verifica a contagem de músicas de cada artista relacionado antes de remover. A associação de gêneros usa createMany com skipDuplicates.

9.3.4 Gêneros

GET /genres usa orderBy name asc. name é único no banco. DELETE exige ausência de relações SongGenre.

9.3.5 Álbuns

releaseDate é convertido para Date e validado. artistIds são normalizados, deduplicados e verificados. Adicionar músicas usa updateMany; remover música do álbum define albumId como null.

9.3.6 Playlists

Músicas são retornadas em position asc. Ao adicionar, a próxima posição é calculada a partir da última. Ao remover, uma transaction exclui a relação e decrementa posições posteriores.

9.3.7 Favoritos

A relação userId + songId é única. Não há UPDATE. GET retorna mais recentes primeiro.

9.3.8 Histórico

secondsListen não pode ser negativo nem exceder duration. minimumSeconds = Math.ceil(duration * 0.25). Abaixo do limiar não há gravação; a resposta é 200 recorded:false. A partir do limiar, 201 recorded:true.

9.3.9 Follows

Há fluxos separados para artista e usuário. Relações compostas impedem duplicatas. No follow de usuários, followerId === followingId é rejeitado com 400.

10. Funcionalidades do sistema

Funcionalidade	O que faz	Endpoints/dados
Cadastro de usuário	Cria usuário com username/email únicos e senha em hash.	POST /users
Edição de perfil	Atualiza nome de usuário, email, senha, bio e imagem.	PATCH /users/:id
Conversão para artista	Cria perfil Artist e primeira música em uma transação.	POST /users/:id/become-artist
Catálogo musical	CRUD de músicas e consultas com artista, álbum e gênero.	/songs
Gêneros	CRUD e associação de múltiplos gêneros por música.	/genres; /songs/:id/genres
Álbuns colaborativos	Álbum pode ter múltiplos artistas e músicas podem ser ligadas/desligadas do álbum.	/albums
Playlists	Criação, edição, privacidade, adição/remoção de músicas e ordem por posição.	/playlists
Favoritos	Usuário favorita e desfavorita músicas; duplicata é impedida.	/users/:id/favorites
Histórico qualificado	Registra reprodução somente após 25% da duração.	/users/:id/history
Follow de artistas	Seguir, listar e deixar de seguir artistas.	following/artists
Follow de usuários	Seguir, listar followers/following e deixar de seguir; auto-follow bloqueado.	following/users
Verificação de artista	Campo verified existe e não é editável na rota pública de artista.	Artist.verified

10.1 Fluxo de playlist ordenada

Fluxo 1 — Reorganização de posições em PlaylistSong

Adicionar músicas:

```
song A -> position 1
song B -> position 2
song C -> position 3
```

Remover song B:

```
1) DELETE PlaylistSong(song B)
2) UPDATE position > 2: decrement 1
```

Resultado:

```
song A -> position 1
song C -> position 2
```

11. Frontend

Não identificado

Não há frontend implementado no estado atual documentado.

React e Vite foram sugeridos no início como stack para o frontend. Também foi discutido que uma landing page com animações poderia ser adicionada posteriormente. Nenhuma página, rota de frontend, componente, hook, estado, formulário, serviço HTTP, navegação ou feedback visual foi implementado no histórico disponível.

Consequentemente, o fluxo “tela → formulário → API → atualização da interface” ainda não existe no projeto. Os testes foram feitos diretamente contra a API via PowerShell.

12. Backend

12.1 Inicialização

Trecho — Responsabilidade central de server.ts

```
import express from "express";

const app = express();
const PORT = 3000;

app.use(express.json());

app.listen(PORT, () => {
  console.log(`Servidor rodando em http://localhost:${PORT}`);
});
```

12.2 Routers

Os routers usam Router() e export default. server.ts importa todos e os registra com app.use(...). A refatoração eliminou acesso direto ao Prisma no server.ts e removeu rotas duplicadas que ainda estavam no arquivo principal.

12.3 Prisma Client

Trecho — src/lib/prisma.ts

```
import "dotenv/config";
import { PrismaPg } from "@prisma/adapter-pg";
import { PrismaClient } from "../generated/prisma/client.js";

const connectionString = process.env.DATABASE_URL!;
const adapter = new PrismaPg({ connectionString });

export const prisma = new PrismaClient({ adapter });
```

12.4 Validação

- Conversão de parâmetros com Number(...).
- Uso de Number.isInteger/Number.isNaN conforme as rotas.
- Checagem de strings obrigatórias e uso de trim em campos textuais específicos.
- Checagem de arrays não vazios para artistIds, songIds, genreIds e artists.
- Verificação prévia de existência de registros relacionados.
- Captura de Prisma P2002 para constraints únicas.

12.5 Camadas

Controller separado: Não identificado no histórico da conversa. Service: Não identificado no histórico da conversa. Repository: Não identificado no histórico da conversa. Models de domínio em classes TypeScript: Não identificado. O schema Prisma representa o modelo persistente.

12.6 Logs

Os blocos catch utilizam console.error(error) e retornam uma mensagem genérica de erro interno. Um sistema estruturado de logging não foi implementado.

13. Autenticação e autorização

13.1 O que existe

Parcialmente implementado	Cadastro e atualização de senha usam bcryptjs para armazenar passwordHash; o hash usa bcrypt.hash(password, 10).
---------------------------	--

13.2 O que não existe ainda

Mecanismo	Estado
Login	Pendente / não implementado
Logout	Não implementado
JWT	Sugestão / não implementado
Cookies	Não identificado no histórico da conversa.
Sessões	Não identificado no histórico da conversa.
Middleware de autenticação	Pendente / não implementado
Controle de propriedade/roles	Pendente / não implementado
Expiração de token	Não se aplica ainda

Hoje, endpoints como `/users/1/favorites` confiam no ID presente na URL. Foi explicitamente identificado como próximo passo substituir essa confiança por uma identidade derivada de autenticação.

14. Segurança

14.1 Mecanismos efetivamente implementados

- Hash de senha com bcryptjs; passwordHash não é retornado nas consultas de usuários.
- DATABASE_URL armazenada em variável de ambiente via .env.
- Validação de entrada em rotas para IDs, tipos, campos obrigatórios e regras de negócio.
- Constraints do banco e Prisma, como @unique e chaves compostas, reforçam integridade.
- Acesso ao banco é feito pelo Prisma Client; nenhum SQL bruto foi identificado.
- verified não é aceito no PATCH público do artista.

14.2 Pontos de atenção de segurança

Sugestão / não implementado	
	Implementar autenticação com login/JWT e middleware para identificar o usuário real da requisição.

- Adicionar autorização por propriedade para impedir que um usuário altere recursos de outro usuário.
- Definir mecanismo administrativo para verified.
- Avaliar CORS quando o frontend for conectado.
- Adicionar rate limiting e política de força/complexidade de senha, caso o projeto evolua para ambiente público.
- Implementar validação centralizada/schemas se o volume de rotas crescer.
- Definir estratégia de upload e validação de arquivos de áudio/imagem antes de habilitar uploads reais.

15. Tratamento de erros

Status	Uso identificado	Exemplos
200	Consulta/sucesso sem criação; histórico abaixo de 25%	GETs; recorded:false; remoção de música de álbum/playlist com mensagem.
201	Criação	Usuário, música, álbum, playlist, favorito, follow, histórico registrado.
204	Exclusão sem corpo	Usuário, artista, música, gênero, álbum, playlist, favorito e unfollow quando permitido.
400	Entrada inválida	ID inválido, body vazio, duração inválida, auto-follow, secondsListen inválido.
404	Recurso inexistente	Usuário, artista, música, gênero, álbum, playlist ou relação não encontrada.
409	Conflito/regra	Duplicatas, artista com músicas, última música do artista, usuário artista não removível.
500	Erro não tratado	catch genérico com console.error(error).

Estrutura genérica de erro 500

```
{
  "message": "Erro interno do servidor"
}
```

Erros de autenticação e autorização não existem porque a autenticação ainda não foi implementada.

16. Configuração do ambiente

16.1 Pré-requisitos confirmados

Item	Versão/estado
Windows	Ambiente utilizado
VS Code	Instalado
Node.js	v24.21.0
npm	11.19.0
TypeScript	7.0.2
PostgreSQL	Instalado
Prisma	7.10.0

16.2 Dependências confirmadas no backend

- express
- bcryptjs
- prisma@7.10.0
- @prisma/client@7.10.0
- @prisma/adapters-pg
- pg
- dotenv
- typescript
- @types/node
- @types/express
- @types/pg

16.3 Configuração TypeScript identificada

Resumo das opções confirmadas de tsconfig.json

```
{
  "compilerOptions": {
    "rootDir": "./src",
    "outDir": "./dist",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "target": "ES2023",
    "types": ["node"],
    "strict": true,
    "esModuleInterop": true,
    "exactOptionalPropertyTypes": true
  },
  "include": ["src/**/*.ts"]
}
```

16.4 Prisma

Generator/datasource consolidados

```
generator client {
  provider      = "prisma-client"
  output        = "../src/generated/prisma"
  moduleFormat  = "esm"
  generatedFileExtension = "ts"
  importFileExtension   = "js"
}

datasource db {
  provider = "postgresql"
}
```

prisma7.config.ts usa dotenv/config, defineConfig, caminho do schema, caminho de migrations e datasource URL obtida de process.env["DATABASE_URL"].

16.5 Como executar o projeto

9. Abrir o terminal em C:\Gouveia_Music\backend.
10. Garantir que PostgreSQL esteja disponível e que DATABASE_URL esteja configurada no .env.
11. Validar o schema com npx prisma validate quando houver mudanças.
12. Aplicar migrations com npx prisma migrate dev --name <nome>.
13. Gerar o client explicitamente com npx prisma generate.
14. Validar TypeScript com npx tsc --noEmit.
15. Compilar com npx tsc.
16. Executar com node .\dist\server.js.
17. Acessar/testar a API em http://localhost:3000.

Comandos confirmados no fluxo de desenvolvimento

```
npx prisma validate
npx prisma migrate dev --name <nome>
npx prisma generate
npx tsc --noEmit
npx tsc
node .\dist\server.js
```

Reset de desenvolvimento também foi utilizado quando necessário: npx prisma migrate reset. Seeds: Não identificado no histórico da conversa.

Frontend: não há instrução de execução porque ele ainda não foi implementado.

17. Testes

17.1 Estratégia de testes

Os testes foram manuais e orientados por endpoint, executados com Invoke-RestMethod no PowerShell. O fluxo padrão foi compilar, iniciar o servidor em um terminal e usar um segundo terminal para chamadas HTTP.

Padrão de execução manual

```
npx tsc
node .\dist\server.js

Invoke-RestMethod `
  -Uri http://localhost:3000/users `
  -Method GET
```

Área	Cobertura manual	Resultado
Users	Criação, listagem, busca, PATCH por campos, PATCH vazio, duplicidade, DELETE, become-artist e bloqueio de DELETE de artista.	Passou
Artists	Listagem, consulta, atualização, PATCH vazio/nome vazio e proteção de DELETE com músicas.	Passou
Songs	Listagem, consulta, criação com artista, artista inexistente, PATCH, PATCH vazio, DELETE e proteção da última música.	Passou
Genres	Criação, ordenação, consulta, duplicidade, PATCH, associação a música, DELETE protegido e DELETE livre.	Passou
Albums	Criação, consulta, PATCH, adicionar/remover música, erro de relação inexistente e DELETE.	Passou após recompilação limpa
Playlists	CRUD, adicionar músicas, duplicata, posições, remoção e reorganização transacional.	Passou
Favorites	Favoritar, listar, duplicata, recurso inexistente, desfavoritar e remover novamente.	Passou
History	Abaixo de 25%, exatamente o limite, acima do limite, listagem, excesso de duração, valor negativo e recurso inexistente.	Passou
Follows	Follow/unfollow artista, duplicata, listas; follow/unfollow usuário, auto-follow, duplicata e listas.	Passou
Servidor/refatoração	Rota raiz, imports, app.use de routers e remoção de rotas antigas do server.ts.	Passou

17.2 Testes automatizados

Não identificado	Nenhum framework de testes automatizados, testes unitários, integração automatizada ou suíte CI foi implementado no histórico.
------------------	--

17.3 Testes de frontend

Não realizados, pois o frontend ainda não existe.

18. Problemas encontrados e soluções

18.1 TypeScript TS18003 — No inputs found

Problema

TypeScript TS18003 — No inputs found

Causa

Configuração include/exclude não encontrava arquivos de entrada.

Solução

Ajuste do tsconfig para incluir os arquivos TypeScript do src.

Impacto

Compilação passou a localizar o código.

18.2 TS1328/TS5024 envolvendo types: [node]

Problema

TS1328/TS5024 envolvendo types: [node]

Causa

Configuração de types estava em formato inválido e/ou tipos Node ainda não estavam instalados.

Solução

Instalação de @types/node e ajuste para types: ["node"].

Impacto

TypeScript passou a reconhecer tipos do Node.

18.3 “This is not the tsc command you are looking for”

Problema

“This is not the tsc command you are looking for”

Causa

O executável TypeScript correto não estava disponível como dependência local.

Solução

Instalação do TypeScript como dependência de desenvolvimento e uso de npx tsc.

Impacto

Comandos de compilação passaram a funcionar.

18.4 Prisma ESM: ERR_MODULE_NOT_FOUND em imports gerados

Problema

Prisma ESM: ERR_MODULE_NOT_FOUND em imports gerados

Causa

O client gerado usava imports sem extensão .js incompatíveis com o runtime ESM compilado.

Solução

Configuração do generator com moduleFormat="esm", generatedFileExtension=".ts" e importFileExtension=".js".

Impacto

Client gerado tornou-se compatível com o projeto ESM.

18.5 Prisma P1012 em UserFollow**Problema**

Prisma P1012 em UserFollow

Causa

Declaração @relation multilinha causou erro de validação no schema usado.

Solução

Relações foram colocadas em uma única linha cada.

Impacto

Schema voltou a validar/migrar.

18.6 Cannot POST /albums**Problema**

Cannot POST /albums

Causa

O processo Node estava executando um dist antigo.

Solução

Parar servidor, recompilar e reiniciar.

Impacto

A rota nova passou a existir.

18.7 Cannot PATCH /artists/1**Problema**

Cannot PATCH /artists/1

Causa

Router novo não estava refletido no servidor em execução/dist.

Solução

Verificação de import/app.use, remoção de rotas antigas e recompilação limpa.

Impacto

PATCH passou a funcionar.

18.8 Cannot PATCH /albums/1 e Cannot DELETE /albums/1/songs/1**Problema**

Cannot PATCH /albums/1 e Cannot DELETE /albums/1/songs/1

Causa

Versão compilada/servidor carregado não tinha as novas rotas.

Solução

Remover dist, npx tsc, confirmar strings no dist e reiniciar Node.

Impacto

Rotas passaram a responder corretamente.

18.9 Exibição “músicas” no PowerShell

Problema

Exibição “músicas” no PowerShell

Causa

Problema visual de encoding no terminal.

Solução

Identificado como exibição do terminal; não houve mudança de regra de negócio registrada.

Impacto

Sem impacto funcional documentado.

18.10 Testes de playlist com recurso ausente

Problema

Testes de playlist com recurso ausente

Causa

Estado do banco durante os testes não tinha playlist/música esperada.

Solução

Criação dos registros necessários antes de repetir o teste.

Impacto

Fluxo de playlist passou a funcionar.

19. Decisões técnicas

Decisão	Contexto/motivo	Consequência	Alternativa/evolução
Node.js + TypeScript + Express para backend	Stack usada durante o desenvolvimento e aprendizado.	Backend tipado e API HTTP simples.	Outras stacks foram discutidas inicialmente, mas esta foi a escolhida.
PostgreSQL + Prisma	Modelagem relacional com migrations e client tipado.	Relacionamentos e constraints centralizados no schema.	Prisma 8 RC foi substituído por Prisma 7.10.0.
Artist como perfil ligado a User	Evitar conta separada para artista.	Artist.userId é unique e usuário pode virar artista.	A alternativa de conta separada foi descartada.
Artist precisa de primeira música	Regra de produto para evitar artista vazio.	become-artist cria Artist + Song + SongArtist em transaction.	-
ListeningHistory em vez de last_song_id	Necessidade de recomendações/retrospectiva mais robustas.	Histórico registra eventos com secondsListen.	Manter apenas última música foi descartado.
Limite de 25% para histórico	Evitar registrar faixas puladas quase imediatamente.	Registro usa Math.ceil(song.duration * 0.25).	-
Músicas podem não ter álbum	Suporte a singles.	Song.albumId é opcional e SetNull ao excluir álbum.	Obrigar álbum foi descartado.
N:N para artistas e gêneros de música	Suporte a colaborações e múltiplos gêneros.	SongArtist e SongGenre.	SongGenre substituiu uma ideia inicial 1:N para gênero.
PlaylistSong com position	Preservar ordem explícita de playlist.	Adição calcula próxima posição; remoção reordena.	-
Verificação do artista controlada pelo sistema	Usuário não deve se autoverificar.	verified não está no PATCH público.	Badge único "Verificado por Gouveia Music".
Refatoração do server.ts	Arquivo cresceu e acumulou rotas.	Routers separados por domínio; server.ts virou composição/configuração.	Manter tudo no server.ts foi abandonado.

20. Fluxos do sistema

20.1 Cadastro de usuário

Fluxo 2 — Cadastro

```

Cliente
-> POST /users
-> valida username/email/password
-> bcrypt.hash(password, 10)
-> prisma.user.create
-> PostgreSQL
-> 201 + usuário sem passwordHash

```

20.2 Tornar-se artista

Fluxo 3 — Conversão para artista

```

User existente
-> POST /users/:id/become-artist
-> valida usuário e dados da primeira música
-> prisma.$transaction
  1. Artist.create
  2. Song.create
  3. SongArtist.create(role="main")
-> 201

```

20.3 Histórico de reprodução

Fluxo 4 — Regra dos 25%

```

POST /users/:id/history
-> busca Song.duration
-> minimumSeconds = ceil(duration * 0.25)
-> secondsListen < minimumSeconds ?
  SIM -> 200 recorded:false (não grava)
  NÃO -> ListeningHistory.create -> 201 recorded:true

```

20.4 Comunicação backend ↔ banco

Fluxo 5 — Persistência

```

Route handler
-> prisma.<model>.<operation>()
-> PrismaPg adapter
-> PostgreSQL gouveia_music
-> resultado tipado
-> response.json()/status().send()

```

20.5 Login/autenticação

Sugestão / não implementado

O fluxo de login/JWT foi planejado como próxima etapa, mas não existe no estado atual.

21. Estado atual do projeto

Item	Estado	Observação
Servidor Express	Implementado e testado	Porta 3000, JSON, routers e fallback 404.
PostgreSQL/Prisma	Implementado e testado	Schema, migrations, client e conexão funcionando.
Usuários	Implementado e testado	CRUD + senha hash + become-artist.
Artistas	Implementado e testado	Read/update/delete protegido; criação via User.
Músicas	Implementado e testado	CRUD + artistas/gêneros/álbum.
Gêneros	Implementado e testado	CRUD + unique + proteção de deleção.
Álbuns	Implementado e testado	CRUD + associação de músicas.
Playlists	Implementado e testado	CRUD + privacidade + position + transaction.
Favoritos	Implementado e testado	Create/read/delete.
Histórico	Implementado e testado	Create/read com regra 25%.
Follow artistas	Implementado e testado	Follow/unfollow e followers.
Follow usuários	Implementado e testado	Follow/unfollow/followers/following.
Autenticação/login	Pendente	bcrypt existe; JWT/login não.
Frontend	Não implementado	React/Vite foi sugerido.
Busca de músicas	Pendente	Mencionada no escopo inicial, sem endpoint implementado.
Player/reprodução real	Pendente	Não existe frontend/player ou streaming real.
Recomendações	Pendente	Histórico prepara dados, mas algoritmo não existe.
Upload de áudio/imagem	Pendente	audioUrl/coverUrl existem como campos; upload não implementado.
Landing page animada	Sugestão / não implementado	Mencionada para fase futura.
Testes automatizados	Não implementado	Testes atuais são manuais.

22. Funcionalidades pendentes

Esta seção contém apenas itens que foram efetivamente mencionados durante a conversa e não concluídos.

Funcionalidade	Situação
Login de usuário	Pendente. Cadastro existe, autenticação não.
JWT e middleware de autenticação	Pendente / próximo passo discutido.
Autorização baseada no usuário autenticado	Pendente. IDs ainda vêm da URL/body.
Busca de músicas	Pendente desde o escopo inicial.
Reprodução/player no frontend	Pendente.
Recomendações por histórico	Pendente; ListeningHistory já fornece base de dados.
Frontend React/Vite	Sugestão / não implementado.
Landing page com animações	Sugestão / não implementado.
Upload real de música e imagens	Pendente; apenas URLs são armazenadas.

23. Próximos passos

23.1 Melhorias sugeridas

- Implementar autenticação e autorização antes de expor a API a usuários reais.
- Separar controller/service/repository somente se a complexidade justificar; atualmente essas camadas não existem.
- Adicionar testes automatizados para regras críticas, especialmente histórico, posições de playlist e exclusões protegidas.
- Centralizar validação de entrada conforme o volume de endpoints crescer.
- Adicionar logging estruturado e configuração diferenciada por ambiente.
- Documentar a API em formato OpenAPI quando a interface estabilizar.

23.2 Novas funcionalidades sugeridas

- Login + JWT.
- Busca de músicas/artistas/álbuns.
- Frontend React/Vite.
- Player musical e integração com audioUrl.
- Upload de áudio/capas.
- Sistema de recomendações usando histórico e gêneros.
- Landing page com animações.

Todos os itens desta seção são sugestões/evoluções futuras e não devem ser interpretados como funcionalidades já existentes.

24. Glossário

Termo	Definição no contexto do projeto
API	Interface HTTP pela qual clientes acessam funcionalidades do backend.
CRUD	Create, Read, Update e Delete.
Endpoint	Combinação de método HTTP e caminho exposta pela API.
Backend	Parte que processa regras, acessa banco e responde à API.
Frontend	Interface visual usada pelo usuário; ainda não implementada.
ORM	Camada que permite manipular banco relacional por objetos/cliente; neste projeto, Prisma.
Router	Objeto Express que agrupa endpoints por domínio.
Middleware	Função executada no fluxo de requisição; atualmente <code>express.json()</code> é o middleware confirmado.
Migration	Mudança versionada do schema do banco aplicada pelo Prisma.
JWT	Token sugerido para autenticação futura; não implementado.
HTTP	Protocolo utilizado entre cliente e API.
REST	Estilo de organização de recursos HTTP; a API atual segue um desenho REST-like.
Transaction	Conjunto de operações executadas atomicamente, usado em <code>become-artist</code> e reordenação de playlist.
Constraint	Regra de integridade do banco, como <code>unique</code> ou chave composta.
Hash de senha	Representação irreversível usada para não armazenar a senha em texto puro; <code>bcryptjs</code> é utilizado.
Prisma Client	Cliente tipado usado pelas rotas para acessar PostgreSQL.
PostgreSQL	Sistema gerenciador de banco de dados relacional do projeto.

25. Conclusão

O Gouveia Music chegou a um backend funcional e testado manualmente, com estrutura relacional capaz de representar usuários, artistas, catálogo musical, gêneros, álbuns, playlists, favoritos, histórico e relacionamentos sociais. O Express recebe as requisições; os routers executam validação e lógica de fluxo; o Prisma Client persiste e consulta os dados em PostgreSQL.

A evolução mais importante de arquitetura foi a passagem de um `server.ts` concentrando rotas para uma estrutura modular em `routes/*.routes.ts`. O schema consolidou relacionamentos N:N por tabelas intermediárias, regras de unicidade e índices. Regras de negócio adicionais são aplicadas na API, como primeira música obrigatória ao virar artista, bloqueio da última música de um artista, histórico após 25% e reordenação transacional de playlists.

O estado atual não inclui frontend, login/JWT, autorização, busca, player real, uploads ou recomendações. Esses itens permanecem como próximos passos já discutidos. A base atual, porém, contém os dados e endpoints necessários para continuar a evolução do projeto sem depender do histórico original da conversa.

A. Apêndice A — server.ts final consolidado

```
// =====  
// IMPORTS  
// =====  
  
import express from "express";  
  
import usersRoutes from "../routes/users.routes.js";  
import artistsRoutes from "../routes/artists.routes.js";  
import songsRoutes from "../routes/songs.routes.js";  
import genresRoutes from "../routes/genres.routes.js";  
import albumsRoutes from "../routes/albums.routes.js";  
import playlistsRoutes from "../routes/playlists.routes.js";  
import favoritesRoutes from "../routes/favorites.routes.js";  
import historyRoutes from "../routes/history.routes.js";  
import followsRoutes from "../routes/follows.routes.js";  
  
const app = express();  
const PORT = 3000;  
  
app.use(express.json());  
  
app.get("/", (request, response) => {  
  response.send("Gouveia Music Backend");  
});  
  
app.use(usersRoutes);  
app.use(artistsRoutes);  
app.use(songsRoutes);  
app.use(genresRoutes);  
app.use(albumsRoutes);  
app.use(playlistsRoutes);  
app.use(favoritesRoutes);  
app.use(historyRoutes);  
app.use(followsRoutes);  
  
app.use((request, response) => {  
  response.status(404).json({  
    message: "Rota não encontrada",  
  });  
});  
  
app.listen(PORT, () => {  
  console.log(`Servidor rodando em http://localhost:${PORT}`);  
});
```

B. Apêndice B — schema.prisma consolidado

```

generator client {
  provider      = "prisma-client"
  output        = "../src/generated/prisma"
  moduleFormat  = "esm"
  generatedFileExtension = "ts"
  importFileExtension   = "js"
}

datasource db {
  provider = "postgresql"
}

model User {
  id            Int           @id @default(autoincrement())
  username      String        @unique
  passwordHash  String
  email         String        @unique
  bio           String?
  profileImageUrl String?
  createdAt     DateTime      @default(now())

  artist      Artist?
  playlists   Playlist[]
  favorites    Favorite[]
  listeningHistory ListeningHistory[]
  followedArtists UserArtistFollow[]
  followingUsers UserFollow[] @relation("UserFollowFollower")
  followers    UserFollow[] @relation("UserFollowFollowing")
}

model Artist {
  id            Int           @id @default(autoincrement())
  userId        Int           @unique
  name          String
  imageUrl      String?
  bio           String?
  verified      Boolean       @default(false)
  createdAt     DateTime      @default(now())

  user          User          @relation(fields: [userId], references: [id], onDelete: Cascade)
  songs         SongArtist[]
  albums        AlbumArtist[]
  followers      UserArtistFollow[]
}

model Song {
  id            Int           @id @default(autoincrement())
  title         String
  duration      Int
  coverUrl      String?
  audioUrl      String
  createdAt     DateTime      @default(now())
  albumId       Int?
  album         Album? @relation(fields: [albumId], references: [id], onDelete: SetNull)
  artists       SongArtist[]
  genres        SongGenre[]
  playlists     PlaylistSong[]
  favorites      Favorite[]
  listeningHistory ListeningHistory[]
}

model Album {
  id            Int           @id @default(autoincrement())
  title         String
  coverUrl      String?
  releaseDate   DateTime
  createdAt     DateTime      @default(now())
  songs         Song[]
  artists       AlbumArtist[]
}

model Genre {
  id            Int           @id @default(autoincrement())

```

```

    name String @unique
    songs SongGenre[]
}

model Playlist {
    id Int @id @default(autoincrement())
    userId Int
    name String
    description String?
    coverUrl String?
    isPublic Boolean @default(false)
    createdAt DateTime @default(now())
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    songs PlaylistSong[]
}

model Favorite {
    id Int @id @default(autoincrement())
    userId Int
    songId Int
    createdAt DateTime @default(now())
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    song Song @relation(fields: [songId], references: [id], onDelete: Cascade)
    @@unique([userId, songId])
}

model ListeningHistory {
    id Int @id @default(autoincrement())
    userId Int
    songId Int
    secondsListen Int
    playedAt DateTime @default(now())
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    song Song @relation(fields: [songId], references: [id], onDelete: Cascade)
    @@index([userId, playedAt])
    @@index([songId])
}

model UserArtistFollow {
    userId Int
    artistId Int
    createdAt DateTime @default(now())
    user User @relation(fields: [userId], references: [id], onDelete: Cascade)
    artist Artist @relation(fields: [artistId], references: [id], onDelete: Cascade)
    @@id([userId, artistId])
    @@index([artistId])
}

model UserFollow {
    followerId Int
    followingId Int
    createdAt DateTime @default(now())
    follower User @relation("UserFollowFollower", fields: [followerId], references: [id],
onDelete: Cascade)
    following User @relation("UserFollowFollowing", fields: [followingId], references: [id],
onDelete: Cascade)
    @@id([followerId, followingId])
    @@index([followingId])
}

model SongArtist {
    songId Int
    artistId Int
    role String
    song Song @relation(fields: [songId], references: [id], onDelete: Cascade)
    artist Artist @relation(fields: [artistId], references: [id], onDelete: Cascade)
    @@id([songId, artistId])
}

model SongGenre {
    songId Int
    genreId Int
    song Song @relation(fields: [songId], references: [id], onDelete: Cascade)
    genre Genre @relation(fields: [genreId], references: [id], onDelete: Cascade)
    @@id([songId, genreId])
}

```

```
model AlbumArtist {
  albumId Int
  artistId Int
  album Album @relation(fields: [albumId], references: [id], onDelete: Cascade)
  artist Artist @relation(fields: [artistId], references: [id], onDelete: Cascade)
  @@id([albumId, artistId])
}

model PlaylistSong {
  playlistId Int
  songId Int
  position Int
  addedAt DateTime @default(now())
  playlist Playlist @relation(fields: [playlistId], references: [id], onDelete: Cascade)
  song Song @relation(fields: [songId], references: [id], onDelete: Cascade)
  @@id([playlistId, songId])
  @@index([playlistId, position])
}
```

C. Apêndice C — comandos de diagnóstico e teste usados

```
# Versões
node -v
npm -v
npx tsc --version

# TypeScript
npx tsc --noEmit
npx tsc
node .\dist\server.js

# Prisma
npx prisma validate
npx prisma migrate dev --name <nome>
npx prisma generate
npx prisma migrate reset

# Diagnóstico de rotas compiladas
Select-String -Path .\src\routes\albums.routes.ts -Pattern 'router.patch|router.delete'
Select-String -Path .\dist\routes\albums.routes.js -Pattern 'router.patch'
Select-String -Path .\dist\server.js -Pattern 'albumsRoutes'

# Recompilação limpa
Remove-Item .\dist -Recurse -Force
npx tsc
node .\dist\server.js
```

Os testes de API foram executados com Invoke-RestMethod, variando métodos GET, POST, PATCH e DELETE e corpos JSON conforme os endpoints documentados no Capítulo 9.

Fim da documentação técnica consolidada do Gouveia Music.